AgriNova — Product & Technical Documentation

AgriNova — Product & Technical Documentation

 This document is a complete description of the AgriNova platform's features, architecture, and data model as of **2026-07-14**, prepared as both onboarding documentation and a dated record of the product's design for intellectual-property purposes. See `PROVENANCE.md` for the authorship/evidence record.

---

1. What AgriNova is

AgriNova is a full-stack web application (Next.js + Supabase) that digitizes the agricultural supply chain in Uganda: it connects smallholder farmers, produce buyers, agri-input suppliers, transporters, crop-disease pathologists, large offtakers, and farmer groups on one platform, with an escrow-based mobile-money payment system (Flutterwave, MTN/Airtel) so that no party has to trust the other with money up front.

It is a **Progressive Web App** — installable, works from a phone browser, no app-store dependency for the initial launch (a native Android build via TWA/Capacitor is a planned follow-up once a Google Play developer account is set up).

Currency: UGX. Localized to Uganda's districts, crops (maize, beans, coffee, banana, cassava, etc.), and mobile-money providers.

2. Roles and what each one can do

The platform is multi-role — a single account can hold more than one role (e.g. a farmer who is also a group member).

Role	Core capabilities
Farmer	List crops for sale, manage farms/soil reports/weather logs, request crop-disease diagnosis (photo or chat with a pathologist), buy inputs from suppliers, request loans (auto farm-score), join a farmer group, receive deliveries, dispute orders, get paid via wallet.
Buyer	Browse/search listings and group lots, place offers or direct orders, fund escrow, track delivery, leave a return/dispute within a 48-hour window, favourite trusted farmers, direct-message farmers.
Transporter	Bid on or accept delivery requests, get matched automatically to open jobs, report live location, get paid on delivery completion.
Supplier	List agricultural inputs (seed, fertilizer, equipment) for farmers to buy, run flash deals with countdown timers, handle farmer-initiated input returns.
Pathologist	Review farmer-submitted disease cases, run/confirm AI diagnoses, hold paid consultations (remote chat or farm visit) with farmers, earn 80% of the consultation fee.
Offtaker	Manage bulk purchase contracts with farmer groups, view scorecards, generate per-contract and consolidated invoices.
Groups (farmer-group admin)	Manage a shared group wallet, list group lots for sale, run group chat, split proceeds among members.

Role	Core capabilities
Admin	Approve/reject listings, review KYC/verification submissions, resolve disputes, configure platform commission, monitor fraud flags, view audit logs.

3. Core systems

3.1 Marketplace

Listings (`listings` table) are posted by farmers or suppliers, browsable by district/crop, and go through an admin approval step (`approval_status`) before appearing publicly. Buyers can make direct offers (negotiated price) or place a direct order at the listed price. Group listings (`group_listings`) let a farmer group sell a pooled lot; proceeds are split via the group wallet.

3.2 Orders, escrow & payments

Every paid transaction is escrow-backed:

1. Buyer and farmer agree a price (direct listing or accepted offer).
2. Buyer funds escrow — money moves from the buyer's wallet into an `escrow_accounts` row, **not** to the farmer yet.
3. A delivery request is auto-created and matched to an available transporter.
4. On delivery confirmation (or after the 48-hour return window closes with no dispute), escrow releases to the farmer, minus platform commission.
5. If the buyer disputes within 48 hours, an admin reviews and resolves — either releasing to the farmer or refunding the buyer.

Wallets (`wallets`) hold a UGX balance per user, funded via Flutterwave mobile-money deposits and cashed out via mobile-money withdrawals. A full ledger (`wallet_transactions`) records every movement. Wallet-to-wallet transfers by account number are supported (`transfer_between_wallets` — see §5).

As of 2026-07-14, every balance-changing operation (deposit, withdrawal, escrow funding, escrow release/refund) runs through an atomic, row-locked database function rather than an application-level read-then-write — see §5 and `docs/PROVENANCE.md` for why that changed.

3.3 Deliveries

A delivery request (`delivery_requests`) is created automatically when escrow is funded, priced by a distance/cargo-based fare calculator (`calcFare`), and offered to available verified transporters (`vehicles.is_available`). A transporter can accept directly or bid; the requester can pick a bid. Live location updates are recorded (`delivery_locations`) during transit. Payment to the transporter happens on delivery completion, split into driver earnings and platform commission.

3.4 Crop disease detection ("Doctor")

Farmers photograph a diseased crop; the image goes to Google Cloud Vision for an automated match against known disease signatures, returning a diagnosis and treatment plan. Farmers can escalate to a human pathologist for a paid consultation (remote chat, UGX 15,000, or a farm visit, UGX 50,000), with real-time chat backed by Supabase Realtime.

3.5 Verification / KYC

Users can apply for one of three trust tiers — green, blue, gold — each requiring more documentation (national ID, selfie, business registration, driving permit, vehicle registration, or professional qualifications depending on role). Documents are uploaded to a **private** Supabase Storage bucket; an admin reviews them via the verification queue and approves/rejects, which updates the user's `verification_level`. Higher tiers unlock features (higher order limits, financing eligibility) and raise the user's visible trust score.

3.6 Loans / financing

A weekly cron job computes a simple farm-score (land size, deal history, crop diversity) per farmer and stores it in `loan_profiles`, which determines the farmer's credit limit for an in-app loan application flow.

3.7 Farmer groups

A group has a shared wallet (`farmer_groups.wallet_balance` / `group_wallet_transactions`), group chat (`group_messages`), and can list pooled produce as a group lot. Proceeds from group sales are split among contributing members.

3.8 Notifications

In-app notifications (`notifications` table) cover payment confirmations, new delivery matches, dispute updates, and admin actions. SMS/email/push integrations are planned but not yet the primary channel — see the launch-readiness checklist in `PROVENANCE.md`.

3.9 Admin console

Admins approve listings, review KYC submissions (with on-demand signed-URL document viewing — documents are never public), resolve disputes, tune platform commission (a percentage with a floor/ceiling fee), and review fraud flags and an audit log of sensitive actions.

4. Technology stack

- **Frontend/Backend**: Next.js (App Router), TypeScript, Tailwind
- **Database/Auth/Storage/Realtime**: Supabase (Postgres + Row Level Security, GoTrue auth, Storage buckets, Realtime channels)
- **Payments**: Flutterwave (MTN Mobile Money, Airtel Money, Uganda)
- **Crop disease AI**: Google Cloud Vision API
- **Weather**: OpenWeatherMap
- **Hosting/CI**: Vercel, deployed via GitHub Actions on push to `main`
- **Validation**: zod (partially applied — see follow-up items)

5. Security & data-integrity model

All authorization is enforced in two layers: application-level checks in each API route (session + ownership + role), **and** Supabase Row Level Security policies at the database layer, so that even a request that bypasses the app (direct Supabase REST/RPC calls with a stolen or self-issued session token) is still constrained by the database itself.

Money movement is never a plain "read balance, compute new balance, write balance" in application code — every path goes through one of these atomic, row-locked Postgres functions (`SECURITY DEFINER` , callable only by the service role):

- `claim_wallet_debit(wallet_id, amount)` — conditional atomic debit, used by withdrawals.
- `credit_wallet(wallet_id, amount)` — atomic credit, used by deposits and refunds.
- `claim_deposit(request_id, user_id, amount, ...)` — atomically claims a mobile-money deposit confirmation and credits the wallet in one step, so a duplicated payment-provider webhook can't double-credit.
- `claim_escrow_fund(order_id, buyer_id, seller_id, amount)` — atomically funds escrow, backed by a unique constraint so a duplicate "fund" request can't double-charge a buyer.
- `transfer_between_wallets(to_account_number, amount, note)` — peer-to-peer wallet transfer with row-level locking on both sides.

A dated, itemized record of what this replaced (and the broader pre-launch security audit that led to it) is kept in `PROVENANCE.md` and is not duplicated here.

6. Data model (core tables)

`profiles` · `farms` · `crops` · `listings` · `group_listings` · `offers` · `orders` · `escrow_accounts` · `wallets` · `wallet_transactions` · `mobile_money_requests` · `delivery_requests` · `delivery_bids` · `delivery_locations` · `vehicles` · `disease_scans` · `disease_cases` · `disease_reports` · `diagnoses` · `consultations` · `direct_messages` · `group_messages` · `farmer_groups` · `group_wallet_transactions` · `supplier_products` · `supplier_orders` · `input_returns` · `offtaker_contracts` · `offtaker_scorecards` · `verifications` · `loan_profiles` · `disputes` · `fraud_flags` · `audit_logs` · `notifications` · `platform_commission` · `buyer_favourites` · `market_prices` / `cash_crop_prices`.

Full column-level detail lives in `supabase/migrations/` (the authoritative, timestamped schema history) and `src/lib/database.types.ts` (generated TypeScript types).

7. What's built vs. what's next

A living build-status record is kept separately from this document (it changes weekly); ask for the current project status rather than assuming this file tracks it. As of this writing, all core flows above are implemented end-to-end. Known follow-ups: application-level rate limiting on auth/payment endpoints, a Content-Security-Policy header, consistent zod validation across the listings/orders/offers/wallet routes, and a native Android wrapper once a Google Play developer account exists.